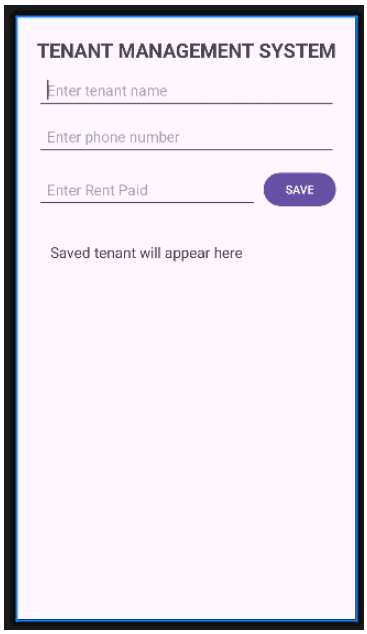
Strathmore UNIVERSITY

SCHOOL OF COMPUTING AND ENGINEERING SCIENCES
BACHELOR IN BUSINESS INFORMATION TECHNOLOGY
BBT 3.2: Mobile Application Development
Language: Kotlin
Lesson 6 Practical — Building the Tenant Management Screen

What we are building

By the end of this practical, you will have a working **Add Tenant** screen:



We will work through:

1. Complete the ConstraintLayout screen
2. Enable View Binding
3. Use View Binding in MainActivity.kt
4. Make the SAVE button work
5. Enable Data Binding
6. Create Tenant.kt
7. Use Data Binding to display the saved tenant

Throughout this guide, boxes marked **Question** answer the things students most often get stuck on. If something goes wrong, check the nearest Question box first, then the **Common errors** table at the end.

PART A — The ConstraintLayout screen

Your project should look something like this:

```
TenantManagementSystem
├── app
├── Gradle Scripts
└── ...
```

Step 1 — Open activity_main.xml

On the left side of Android Studio, make sure the Project panel is set to **Android** view. Then click:

```
app
 └── res
     └── layout
         └── activity_main.xml
```

Double-click activity_main.xml to open it. In the top-right corner of the editor, click **Code** so you can see the XML.

Step 2 — Replace the contents of activity_main.xml

If your file does not already contain the following layout, delete the existing XML and replace it with:

```
<?xml version="1.0" encoding="utf-8"?>

<androidx.constraintlayout.widget.ConstraintLayout
   xmlns:android="http://schemas.android.com/apk/res/android"
   xmlns:app="http://schemas.android.com/apk/res-auto"
   android:layout_width="match_parent"
   android:layout_height="match_parent">

   <TextView
      android:id="@+id/titleTextView"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="TENANT MANAGEMENT SYSTEM"
      android:textSize="24sp"
      android:textStyle="bold"
      android:layout_marginTop="24dp"
      app:layout_constraintTop_toTopOf="parent"
      app:layout_constraintStart_toStartOf="parent"
      app:layout_constraintEnd_toEndOf="parent" />

   <TextView
      android:id="@+id/tenantNameLabel"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="Tenant Name"
      android:textSize="16sp"
      android:layout_marginTop="24dp"
      android:layout_marginStart="24dp"
      app:layout_constraintTop_toBottomOf="@id/titleTextView"
      app:layout_constraintStart_toStartOf="parent" />

   <EditText
      android:id="@+id/tenantNameEditText"
      android:layout_width="0dp"
      android:layout_height="wrap_content"
      android:hint="Enter tenant name"
      android:minHeight="48dp"
      android:padding="12dp"
      android:layout_marginTop="8dp"
```

```
    android:layout_marginStart="24dp"
    android:layout_marginEnd="24dp"
    app:layout_constraintTop_toBottomOf="@id/tenantNameLabel"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent" />

<TextView
    android:id="@+id/phoneLabel"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Phone Number"
    android:textSize="16sp"
    android:layout_marginTop="16dp"
    android:layout_marginStart="24dp"
    app:layout_constraintTop_toBottomOf="@id/tenantNameEditText"
    app:layout_constraintStart_toStartOf="parent" />

<EditText
    android:id="@+id/phoneEditText"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:hint="Enter phone number"
    android:inputType="phone"
    android:minHeight="48dp"
    android:padding="12dp"
    android:layout_marginTop="8dp"
    android:layout_marginStart="24dp"
    android:layout_marginEnd="24dp"
    app:layout_constraintTop_toBottomOf="@id/phoneLabel"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent" />

<EditText
    android:id="@+id/rentEditText"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:hint="Enter Rent Paid"
    android:inputType="numberDecimal"
    android:minHeight="48dp"
    android:padding="12dp"
    android:layout_marginStart="24dp"
    android:layout_marginTop="16dp"
    android:layout_marginEnd="8dp"
    app:layout_constraintTop_toBottomOf="@id/phoneEditText"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toStartOf="@id/saveButton" />

<Button
    android:id="@+id/saveButton"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="SAVE"
    android:layout_marginEnd="24dp"
    app:layout_constraintTop_toTopOf="@id/rentEditText"
    app:layout_constraintEnd_toEndOf="parent" />

<TextView
    android:id="@+id/tenantResultTextView"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:text="Saved tenant will appear here"
    android:textSize="18sp"
    android:padding="16dp"
    android:layout_margin="24dp"
    app:layout_constraintTop_toBottomOf="@id/rentEditText"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent" />
```

```
</androidx.constraintlayout.widget.ConstraintLayout>
```

Run the application. You should see the static screen: a title, three input fields and a SAVE button. Tapping SAVE does nothing yet.

> **Question: Why does the rent field use `inputType="numberDecimal"`?**
> So the phone shows a number keyboard instead of a full letter keyboard. The phone field uses `inputType="phone"` for the same reason.
>
> **Question: Why are the EditText widths `0dp`?**
> In a ConstraintLayout, `0dp` means "match constraints": stretch between the start and end constraints (minus the margins). It does not mean zero width.

PART B — View Binding

We have created the interface. Now we need Kotlin to communicate with the views.

Previously, you may have used `findViewById()`. In this practical we use **View Binding**, which gives us a safer and shorter way to reach every view that has an ID.

Step 3 — Open the app-level Gradle file

On the left, click the arrow beside:

```
Gradle Scripts
```

You will see two files with similar names:

```
build.gradle.kts (Project: TenantManagementSystem)
build.gradle.kts (Module :app)
```

Open **build.gradle.kts (Module :app)**. Do not open the Project one.

> **Question: Which one? They look the same.**
> Always the one that says **Module :app**. The Project file configures the whole project; the Module file configures your app, and that is where features like View Binding are turned on. If you edit the wrong one, you will see an error that `android` or `buildFeatures` is not recognised.

Step 4 — Find android { } and enable View Binding

Scroll down until you see something similar to:

```
android {
    namespace = "com.example.tenantmanagementsystemgroupa"
    compileSdk = ...


    ...
}
```

Inside the `android { }` block, add:

```
    buildFeatures {
        viewBinding = true
    }
```

For example:

```
android {

    namespace = "com.example.tenantmanagementsystemgroupa"

    compileSdk = ...

    buildFeatures {
        viewBinding = true
    }

    ...
}
```

Do not put it outside `android { }`.

> **Question: Can I put it outside `android { }`?**
> No. `buildFeatures` only exists inside `android { }`. Outside it, you will get `Unresolved reference: buildFeatures`.
>
> **Question: I already have a `buildFeatures` block.**
> Do not create a second one. Add `viewBinding = true` inside the existing block.
>
> **Question: My package name is different from the one in this guide.**
> That is fine. Everywhere this guide shows `com.example.tenantmanagementsystemgroupa`, use your own package name. You can find it on the first line of `MainActivity.kt`.

Step 5 — Sync the project

After adding the code, Android Studio shows a yellow bar at the top of the editor:

```
Gradle files have changed since last project sync.   Sync Now
```

Click **Sync Now** and wait for Gradle to finish (watch the progress bar at the bottom of the window).

> **Question: The yellow bar disappeared before I clicked it.**
> Use **File → Sync Project with Gradle Files**, or click the elephant icon with the blue arrow in the toolbar.

PART C — Use View Binding in MainActivity

Now we connect our XML to Kotlin.

Step 6 — Open MainActivity.kt

On the left:

```
app
└── kotlin+java
    └── com.example.tenantmanagementsystemgroupa
        └── MainActivity.kt
```

Double-click `MainActivity.kt`.

Step 7 — Import the binding class

At the top of `MainActivity.kt`, with the other `import` lines, add:

```
import com.example.tenantmanagementsystemgroupa.databinding.ActivityMainBinding
```

Your package name may be different. The important part is the ending:

```
.databinding.ActivityMainBinding
```

Why ActivityMainBinding? Because our XML file is called activity_main.xml. Android generates a binding class for every layout using this rule: remove the underscores, capitalise each word, and add Binding.

```
activity_main.xml  →  ActivityMainBinding
```

> **Question: ActivityMainBinding is red.**
> Check three things: (1) you clicked Sync Now in Step 5; (2) the import uses *your* package name followed by .databinding; (3) if both are correct, use **Build → Rebuild Project**. You can also click the red word and press **Alt+Enter** (Mac: **Option+Enter**) to add the correct import automatically.

Step 8 — Create the binding variable

Inside MainActivity, but before onCreate(), add:

```
private lateinit var binding: ActivityMainBinding
```

So it looks like this:

```
class MainActivity : AppCompatActivity() {

    private lateinit var binding: ActivityMainBinding

    override fun onCreate(savedInstanceState: Bundle?) {
```

> **Question: What does lateinit mean?**
> It means "I promise to give this variable a value before I use it." We cannot create the binding until onCreate() runs, so we declare it here and fill it in inside onCreate().

Step 9 — Inflate the binding

Inside onCreate(), add:

```
binding = ActivityMainBinding.inflate(layoutInflater)
```

Then replace:

```
setContentView(R.layout.activity_main)
```

with:

```
setContentView(binding.root)
```

The beginning of your Activity becomes:

```
class MainActivity : AppCompatActivity() {

    private lateinit var binding: ActivityMainBinding

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        binding = ActivityMainBinding.inflate(layoutInflater)

        setContentView(binding.root)
    }
}
```

> **Question: My MA inactivity also has enableEdgeToEdge() and a block starting with ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main)). Do I keep it?**
> Delete those lines for this practical. That code looks for a view with the ID main, and our layout does not have one, so the app would crash when it opens.

Question: What is binding.root?
The outermost view in the layout, which here is the ConstraintLayout. setContentView(binding.root) shows the whole layout on screen, just like setContentView(R.layout.activity_main) did before.

Step 10 — Access the views

We no longer need findViewById(). Every view that has an ID is now available through binding.

Because the XML contains:

```
android:id="@+id/tenantNameEditText"
```

Android creates:

```
binding.tenantNameEditText
```

Similarly:

```
binding.phoneEditText
binding.rentEditText
binding.saveButton
binding.tenantResultTextView
```

Question: What happens to a view that has no ID?
It does not appear in the binding class. Only views with an android:id can be reached through binding.

Question: If my ID is tenant_name_edit_text, what is it called in binding?
binding.tenantNameEditText. Underscores are removed and the words are joined in camelCase.

Question: Is View Binding the same as findViewById?
It does the same job, but more safely. With findViewById, a wrong ID still compiles and then crashes when the app runs. With View Binding, a wrong ID shows a red error before you even run the app.

Step 11 — Make SAVE work

Inside onCreate(), underneath setContentView(binding.root), add:

```
binding.saveButton.setOnClickListener {

    val name = binding.tenantNameEditText.text.toString()

    val phone = binding.phoneEditText.text.toString()

    val rent = binding.rentEditText.text.toString()

    binding.tenantResultTextView.text =
        "Tenant: $name\nPhone: $phone\nRent: KSh $rent"
}
```

Your complete MainActivity.kt should now look similar to:

```
package com.example.tenantmanagementsystemgroupa

import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity
import com.example.tenantmanagementsystemgroupa.databinding.ActivityMainBinding

class MainActivity : AppCompatActivity() {

    private lateinit var binding: ActivityMainBinding

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        binding = ActivityMainBinding.inflate(layoutInflater)
```

```
        setContentView(binding.root)

        binding.saveButton.setOnClickListener {

            val name = binding.tenantNameEditText.text.toString()

            val phone = binding.phoneEditText.text.toString()

            val rent = binding.rentEditText.text.toString()

            binding.tenantResultTextView.text =
                "Tenant: $name\nPhone: $phone\nRent: KSh $rent"
        }
    }
}
```

Question: What does `\n` do?
It starts a new line inside the text, so name, phone and rent appear on separate lines.

Question: What does `$name` do inside the quotes?
It inserts the value of the variable name into the text. This is called a string template.

Step 12 — Run the application

Enter:

```
Tenant Name:  John Kamau
Phone:        0712345678
Rent:         25000
```

Click **SAVE**. The screen should show:

```
Tenant: John Kamau
Phone: 0712345678
Rent: KSh 25000
```

Checkpoint. You have now connected everything in this chain:

```
XML
↓
View Binding
↓
Kotlin
↓
User interaction
↓
Updated UI
```

PART D — Enable Data Binding

View Binding and Data Binding are related, but they solve slightly different problems.

	View Binding	Data Binding
What it does	Lets **Kotlin** reach the views	Lets the **XML** read data directly
Direction	Code → views	Data → views
Layout change needed	None	The layout is wrapped in a `<layout>` tag
Build speed	Fast	Slower (needs the kapt plugin)

	View Binding	Data Binding
Typical use	Most screens	Forms and screens that display objects

Look at Step 11 again. Kotlin does two jobs there: it collects the input, **and** it decides exactly how the result text looks ("Tenant: $name\nPhone: ..."). With Data Binding, Kotlin only hands over a **Tenant object**, and the XML decides how to display it.

```
View Binding:   Kotlin ──builds the text──▶ TextView

Data Binding:   Kotlin ──gives a Tenant──▶ XML ──displays it──▶ TextView
```

Step 13 — Add the kapt plugin

Open **build.gradle.kts (Module :app)** again.

Scroll to the very **top** of the file. You will see a plugins { } block, similar to:

```
plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.kotlin.android)
}
```

Add one line inside it:

```
plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.kotlin.android)
    id("kotlin-kapt")
}
```

> **Question: What is kapt?**
> A tool that generates code while your app is building. Data Binding needs it to create the code that connects your XML to your Kotlin data.
>
> **Question: The other plugins use alias(...) and this one uses id(...). Is that allowed?**
> Yes. They are two different ways of naming a plugin, and you can mix them.

Step 14 — Turn on Data Binding

In the same file, find:

```
buildFeatures {
    viewBinding = true
}
```

and change it to:

```
buildFeatures {
    viewBinding = true
    dataBinding = true
}
```

Click **Sync Now**.

> **Question: After syncing, I see a warning about kapt and Kotlin language version 2.0.**
> That is a warning, not an error. The project will still build and run.
>
> **Question: Do I need both viewBinding and dataBinding?**
> You can keep both. We still use binding.saveButton and the EditTexts exactly as before, and Data Binding adds the ability for the XML to display data.

PART E — Create the Tenant class

Data Binding needs an object to display. We create a class that represents one tenant.

Step 15 — Create Tenant.kt

On the left, right-click:

```
com.example.tenantmanagementsystemgroupa
```

Then choose:

```
New → Kotlin Class/File
```

Type the name:

```
Tenant
```

Do not type .kt; Android Studio adds it automatically. Choose **Class** and press Enter.

Your package should now contain:

```
com.example.tenantmanagementsystemgroupa
├── MainActivity.kt
└── Tenant.kt
```

> **Question: I right-clicked and only see "New → File" or "Package".**
> Make sure you right-clicked the **package** (com.example.tenantmanagementsystemgroupa), not the kotlin+java folder or the res folder.

Step 16 — Write the Tenant class

Open Tenant.kt and replace its contents with:

```
package com.example.tenantmanagementsystemgroupa

data class Tenant(
    val name: String,
    val phone: String,
    val rent: String
) {
    fun summary(): String {
        return "Tenant: $name\nPhone: $phone\nRent: KSh $rent"
    }
}
```

The three properties match the three fields on our screen: name, phone and rent.

> **Question: What does data class mean?**
> A class whose main job is to hold data. Kotlin writes useful code for you automatically, for example a readable toString(), so printing a tenant shows its values.
>
> **Question: What is summary() for?**
> It describes how a tenant should be shown as text. The XML will call it in Step 19. Notice it's the same text we built inside MainActivity in Step 11, but now it belongs to the Tenant.
>
> **Question: Why is rent a String and not a number?**
> Because for now we only display it. When we start calculating totals on the dashboard, we will store it as a number.

PART F — Use Data Binding in the layout

Step 17 — Convert activity_main.xml to a data binding layout

Open activity_main.xml and make sure you are in **Code** view.

Click anywhere on the word:

```
androidx.constraintlayout.widget.ConstraintLayout
```

on the **first** ConstraintLayout line (the opening tag near the top, not the closing tag at the bottom).

Press **Alt+Enter** (Mac: **Option+Enter**) and choose:

```
Convert to data binding layout
```

Your file now starts like this:

```
<?xml version="1.0" encoding="utf-8"?>
<layout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto">

    <data>

    </data>

    <androidx.constraintlayout.widget.ConstraintLayout
        android:layout_width="match_parent"
        android:layout_height="match_parent">
```

and the very last line of the file is now:

```
</layout>
```

> **Question: I don't see "Convert to data binding layout".**
> Make sure your cursor is on the root ConstraintLayout tag, not on a TextView inside it. Also check you clicked Sync Now in Step 14. If it still doesn't appear, you can type the <layout> and <data> tags by hand: move the two xmlns: lines from the ConstraintLayout up to <layout>, and add </layout> at the very end.
>
> **Question: Where did xmlns:android and xmlns:app go?**
> They moved up to <layout>. <layout> is now the outermost tag, and those declarations must be on the outermost tag.
>
> **Question: What is <data> for?**
> It is where you list the objects the layout is allowed to use. Right now it is empty. We fill it in the next step.

Step 18 — Declare the tenant variable

Inside the empty <data> </data>, add:

```
    <data>
        <variable
            name="tenant"
            type="com.example.tenantmanagementsystemgroupa.Tenant" />
    </data>
```

This tells the layout: "You can use a Tenant object, and inside this XML it is called tenant."

> **Question: What is the difference between name and type?**
> name is what we call the object inside the XML (tenant). type is the full address of the class: your package name plus the class name. It must match **your** package exactly.

Step 19 — Bind the result TextView to the tenant

Scroll down to tenantResultTextView. Find this line:

```
android:text="Saved tenant will appear here"
```

Replace it with these two lines:

```
android:text="@{tenant.summary()}"
android:hint="Saved tenant will appear here"
```

Your TextView now looks like this:

```
<TextView
    android:id="@+id/tenantResultTextView"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:text="@{tenant.summary()}"
    android:hint="Saved tenant will appear here"
    android:textSize="18sp"
    android:padding="16dp"
    android:layout_margin="24dp"
    app:layout_constraintTop_toBottomOf="@id/rentEditText"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent" />
```

Question: What does @{ } mean?
It is a Data Binding expression. Anything inside @{ } is read from your data. @{tenant.summary()} means "call summary() on the tenant, and show the result here."

Question: Why did we change the text to a hint?
When the app first opens, no tenant has been saved yet, so there is nothing to display. A hint is grey placeholder text that shows only when the TextView is empty. As soon as a tenant is saved, the real text replaces it.

Question: Won't tenant.summary() crash when there is no tenant yet?
No. Data Binding is null-safe. If tenant is empty, the expression simply produces no text, and the hint is shown.

Question: The design view no longer shows "Saved tenant will appear here" in black.
It shows it in grey now, because it's a hint. The real text only exists when the app runs.

PART G — Give the layout a Tenant from Kotlin

Step 20 — Update the SAVE code

Open MainActivity.kt. Find the SAVE block from Step 11.

Replace these lines:

```
binding.tenantResultTextView.text =
    "Tenant: $name\nPhone: $phone\nRent: KSh $rent"
```

with:

```
val tenant = Tenant(name, phone, rent)

binding.tenant = tenant
```

Your complete MainActivity.kt:

```
package com.example.tenantmanagementsystemgroupa

import android.os.Bundle
```

```
import androidx.appcompat.app.AppCompatActivity
import com.example.tenantmanagementsystemgroupa.databinding.ActivityMainBinding

class MainActivity : AppCompatActivity() {

    private lateinit var binding: ActivityMainBinding

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        binding = ActivityMainBinding.inflate(layoutInflater)

        setContentView(binding.root)

        binding.saveButton.setOnClickListener {

            val name = binding.tenantNameEditText.text.toString()

            val phone = binding.phoneEditText.text.toString()

            val rent = binding.rentEditText.text.toString()

            val tenant = Tenant(name, phone, rent)

            binding.tenant = tenant
        }
    }
}
```

Question: Where does `binding.tenant` come from?
From `<variable name="tenant" .../>` in the XML (Step 18). Every variable you declare in `<data>` becomes a property on the binding class.

Question: `binding.tenant` is red.
Rebuild the project (**Build → Rebuild Project**) so Android regenerates `ActivityMainBinding` with the new variable. If it is still red, check that the `<variable>` name in the XML is exactly `tenant`.

Question: Do I still use `ActivityMainBinding.inflate(...)` now that it's Data Binding?
Yes. The same line works for both. `ActivityMainBinding` now has everything from View Binding (the views) plus the `tenant` variable from Data Binding.

Question: Why is this better than Step 11?
In Step 11, MainActivity decided how a tenant looks on screen. Now MainActivity only creates the Tenant and hands it over. How a tenant is displayed lives in the Tenant class and the XML. When the app grows, the same Tenant can be shown on other screens without rewriting that text each time.

Step 21 — Run the application

Click **Run ▶**.

1. When the app opens, the result area shows the grey hint: *Saved tenant will appear here.*
2. Enter:

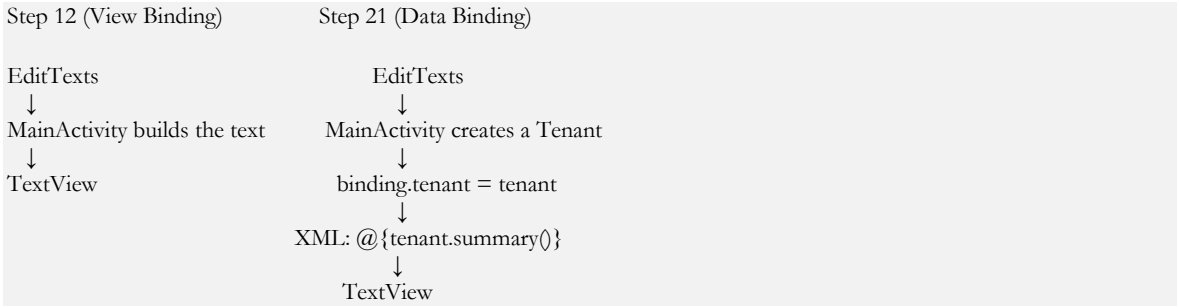
```
Tenant Name:  John Kamau
Phone:        0712345678
Rent:         25000
```

1. Click **SAVE**. The screen shows:

```
Tenant: John Kamau
Phone: 0712345678
Rent: KSh 25000
```

1. Change the name to Mary Wanjiku and click **SAVE** again. The result updates.

The screen looks the same as in Step 12, but the way it works is different:



Final project structure



Common errors

What you see	Cause	Fix
Unresolved reference: buildFeatures	Added outside android { }, or in the Project Gradle file	Move it inside android { } in **Module :app**
ActivityMainBinding is red	Not synced, or wrong package in the import	Sync, then Alt+Enter on the red word
App crashes on launch after editing MainActivity	Leftover ViewCompat...R.id.main code	Delete the enableEdgeToEdge() and ViewCompat... lines
R or Tenant is red in a Kotlin file	Package line does not match your app's package	Fix the package line at the top of the file
No "Convert to data binding layout" option	Cursor not on the root tag, or not synced after Step 14	Click the opening ConstraintLayout tag; sync again
Build fails: Cannot resolve type 'Tenant'	The type in <variable> has the wrong package	Copy your package name exactly from Tenant.kt
Build fails: Could not find identifier 'tenant'	The name inside @{ } does not match <variable name>	Make both exactly tenant
binding.tenant is red	Binding class not regenerated	**Build → Rebuild Project**

What you see	Cause	Fix
Tapping SAVE shows nothing	binding.tenant = tenant is missing	Add it inside the SAVE block (Step 20)

Try it yourself

1. If the name field is empty when SAVE is tapped, show an error on the field instead of saving. Hint: binding.tenantNameEditText.error = "Tenant name is required", then return@setOnClickListener.
2. Change summary() in Tenant.kt so the rent shows as Rent paid: KSh 25000. Run the app. Notice that you did not touch MainActivity.kt at all.
3. Add a second TextView to the layout that shows only the tenant's name in large bold text. Hint: android:text="@{tenant.name}".
4. After saving, clear the three input fields. Hint: binding.tenantNameEditText.text.clear().

What you learned today

```
STEP 1   ConstraintLayout
       "I can design my screen."
             ↓
STEP 2   View Binding
       "I can access my XML views from Kotlin."
             ↓
STEP 3   Data Binding
       "I can give my XML an object and let it display the data."
```

Coming up next

The screen you built today is the **Add Tenant** screen. Next, we build a separate **View Tenants** screen that shows all tenants in a scrolling list using a RecyclerView, and then connect all the screens of the app with App Navigation:

